

These are the instructions to you to how to learn data visualization using this question

Firstly, you should carefully read the problem in the problem PDF. Then, you should try to answer the questions one by one. This PDF is designed for you to learn data visualization and data analysis by answering these questions. To do that, read the question carefully—for example, if you are going to answer question one, read it carefully first. Then, go to my Python code. In that code, you can see detailed information on how to answer question one, what functions you should use, and why you should use them, with every detail you should know.

After reading those explanations carefully, try to answer the questions by yourself. Then, come back to my Python file and check your work against my answers. For some questions, you will see a note saying "please refer to the Read Me Please PDF." This is the same PDF that you are now reading. On the relevant pages, you may see instructions and solutions for those questions. Read those solutions and functions, learn what you can do using those commands, and then answer the question. After that, try to compare your code with my answers.

Disclaimer:

Please be aware that I am also a learner, so my codes are not perfect, and I don't claim that they are the best. I created these notes and this PDF while learning machine learning to help other students. When I was learning machine learning, the biggest problem I encountered was that most machine learning lecturers just write the code instead of explaining what we can actually do with it or with a specific command. To overcome that, I had to do my own research and figure things out myself.

To make everyone's work easier, I gathered all of those details within this project to help you on your machine learning journey. I don't guarantee that you will become super smart after reading this—maybe you are already familiar with this topic, and if so, you can simply ignore it. But if you are a learner or a beginner, I believe this will be really helpful to you.

Also, please note that these are not the only methods to solve these problems. There are thousands of methods, and I don't claim mine are the best or the easiest. If there are any other methods, please feel free to share them with me!

To answer **question six**, to draw the graph for question six, we need to figure out which column to use, and specifically and carefully we should use the data in the timestamp column. But before that, we have to make sure that that column has the correct data type. If you run step one, you may figure out that in that column, it has the string values. So, we have to implement step two to turn those string values into proper datetime values. And then in step three, we create new columns in this dataset. By using those columns we can draw the graphs for question 6

Step 1: Checking the Data Type

```
type(df['timeStamp'].iloc[0])
```

- `df['timeStamp']`: Targets the timestamp column.
- `.iloc[0]`: Selects the very first row (index 0) of that column.
- `type(...)`: Checks the Python data type of that specific value.
- **Why we do it / What it means**: When data is loaded fresh from a CSV file, dates and times are treated as plain text strings (`str`), not actual calendar dates. Running this returns `str`, proving that Python currently sees the timestamps as simple text characters rather than dates.

Step 2: Converting Strings to Datetime Objects

```
df['timeStamp'] = pd.to_datetime(df['timeStamp'])
```

- `pd.to_datetime(...)`: A powerful pandas function that reads text strings formatted like dates and times (e.g., "2015-12-10 17:10:52") and converts them into official Datetime objects.
- **Why we do it / What it means**: Once a column is converted into true datetime objects, Python "unlocks" special built-in time attributes. Instead of just seeing letters and numbers, Python now recognizes years, months, days, hours, minutes, and days of the week.

Step 3: Creating New Time-Feature Columns

```
df['Hour'] = df['timeStamp'].dt.hour
df['Month'] = df['timeStamp'].dt.month
df['Day of Week'] = df['timeStamp'].dt.dayofweek

# Map integer days to text names using your dictionary
dmap = {0:'Mon', 1:'Tue', 2:'Wed', 3:'Thu', 4:'Fri', 5:'Sat', 6:'Sun'}
df['Day of Week'] = df['Day of Week'].map(dmap)
```

Step-by-Step Breakdown of What the Code Does

1. Building the Matrix (`dayHour`)

- `df.groupby(by=['Day of Week', 'Hour'])`: Groups your data simultaneously by two columns—first splitting by the day of the week, and then splitting further by each hour of the day.
- `.count()['Reason']`: Counts how many emergency calls occurred during each specific hour of each specific day.
- `.unstack()`: This is the magic method! It takes a hierarchical multi-index table and pivots the inner index (`Hour`) out into horizontal columns. This converts your long vertical list into a clean, 2D matrix table where rows are days, columns are hours, and the inner cells are the counts.

2. Rendering the Heatmap (`sns.heatmap`)

- `sns.heatmap(dayHour, ...)`: A specialized Seaborn function that takes a 2D matrix table and paints every cell with a color proportional to its numerical value (higher counts get bright yellow/green, lower counts get dark purple).
- `cmap='viridis'`: Defines the color theme (colormap) for the heatmap. `'viridis'` transitions smoothly from dark purple to bright yellow, providing high contrast and looking very professional.

Would you like to explore creating a similar heatmap comparing **Days of the Week vs. Months**, or are you ready to move on to the next section of your project?